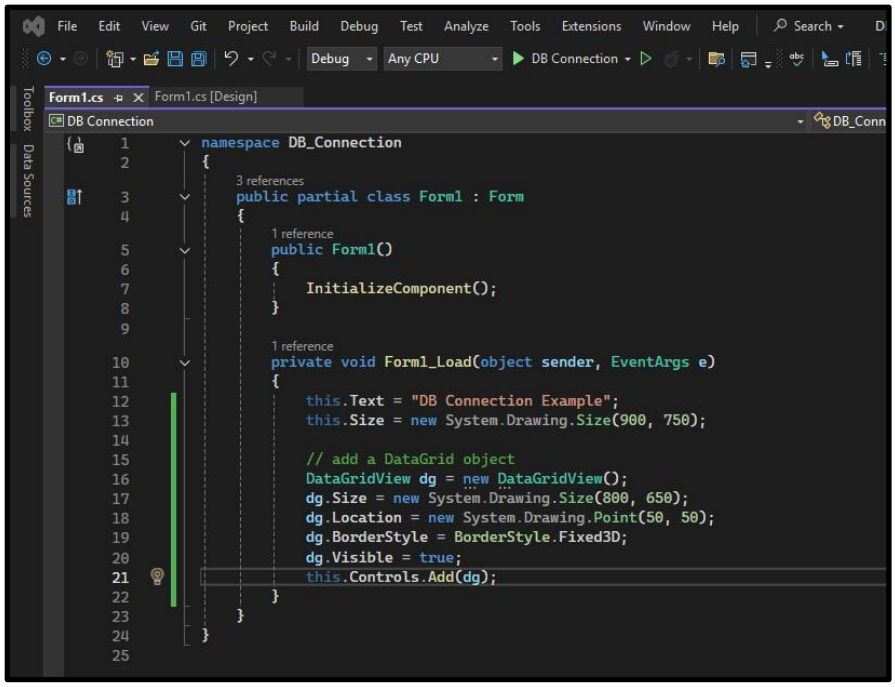
Chapter 20: Database Connection in C#

In our last chapter we took a look at the SQL syntax. Now it's time to learn how to execute some SQL from within our C# applications. The good news is that Microsoft has provided us with a very nice set of classes for realizing this feat.

Let's jump to the inside and learn how to make C# and SQL play nice together.

We begin by just setting up our form. (Figure 20.1) The only interesting thing here is this DataGridView class. This guy is going to provide us with a row by column matrix for displaying and editing data. All you really need to do is set the DataSource property of this guy to a set of rows from an SQL query and you are ready to go.

Figure 20.1: Form Setup

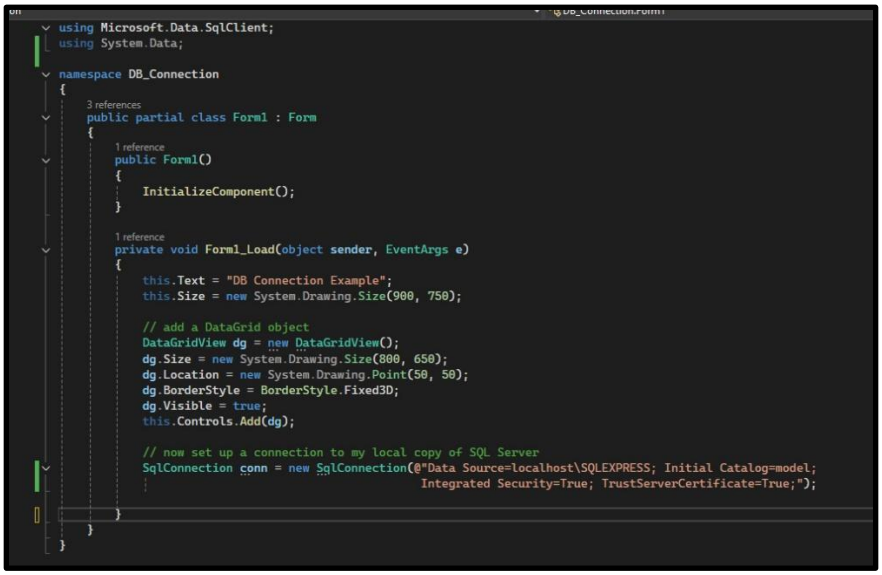


This next image is a little more interesting. The goal here is to create a connection to my local copy of SQL Server. The class we use here is SQLConnection and that guy lives in the Microsoft.Data.SQLClient namespace. We of course begin by adding the "*using Microsoft.Data.SQLClient*" namespace to our code. Unfortunately, this namespace is not included in the standard Visual Studio install so you need to go get it. To do this go to the Project top menu option and then scroll down to the Manage nuGet Packages option. Now you can just do a search on the namespace, i.e. Microsoft.Data.SQLClient, and then finally do the install.

Once you have that namespace installed and referenced in your code you can create a connection string to the database using the SQLConnection class as shown here. (Figure 20.2)

A couple of things about this connection string warrant discussion. First, the *Data Source = local host\SQLEXPPRESS* syntax means that this database is running on the local host and we are using the SQLExpress edition of the software. If you are using a different configuration you will need to adjust your syntax accordingly. Next, the *Initial Catalog = model* syntax says the connection will utilize the model schema. The *Integrated Security = True* parameter says we will be using standard Windows authentication. Again, if you are using something other than Windows you will need to check with Google as to your necessary syntax. Finally, the *TrustedServerCertificate = True* syntax says the connection will utilize SSL to encrypt the data transport channel.

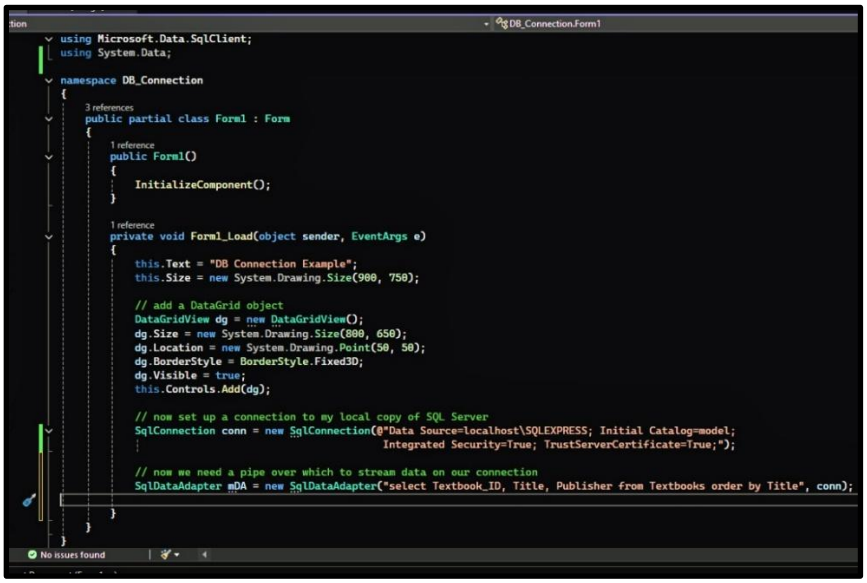
Figure 20.2: SQLConnection Class



```
using Microsoft.Data.SqlClient;
using System.Data;

namespace DB_Connection
{
    3 references
    public partial class Form1 : Form
    {
        1 reference
        public Form1()
        {
            InitializeComponent();
        }

        1 reference
        private void Form1_Load(object sender, EventArgs e)
        {
            this.Text = "DB Connection Example";
            this.Size = new System.Drawing.Size(900, 750);

            // add a DataGrid object
            DataGridView dg = new DataGridView();
            dg.Size = new System.Drawing.Size(800, 650);
            dg.Location = new System.Drawing.Point(50, 50);
            dg.BorderStyle = BorderStyle.Fixed3D;
            dg.Visible = true;
            this.Controls.Add(dg);

            // now set up a connection to my local copy of SQL Server
            SqlConnection conn = new SqlConnection(@"Data Source=localhost\SQLEXPRESS; Initial Catalog=model;
                                                   Integrated Security=True; TrustServerCertificate=True;");

        }
    }
}
```

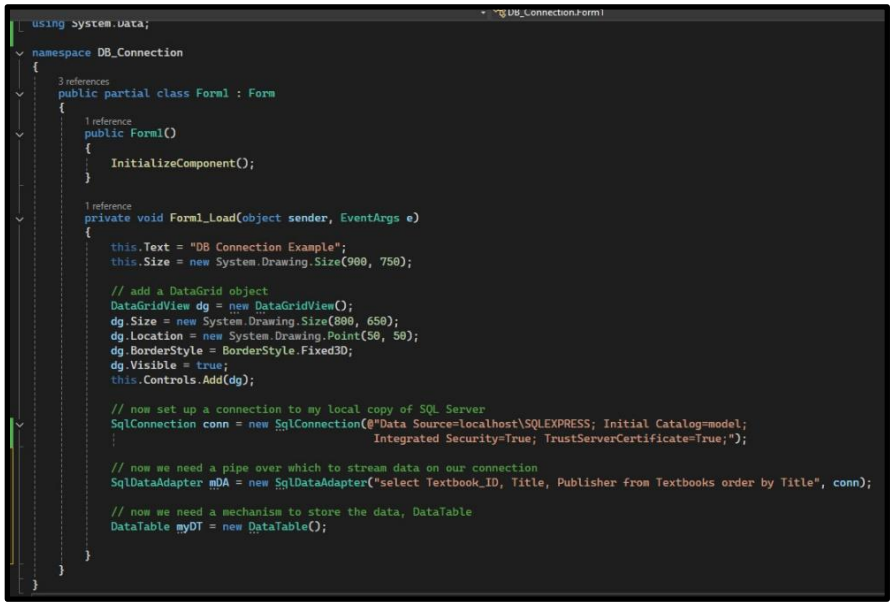
Now that we have a database connection established we need to create a pipe over which we will stream data to and from our schema. The class here is named SQLAdapter and the most typical usage takes two parameters. The first parameter is a piece of SQL and the second parameter is the name of your connection class. (Figure 20.3)

Figure 20.3: SQLAdapter



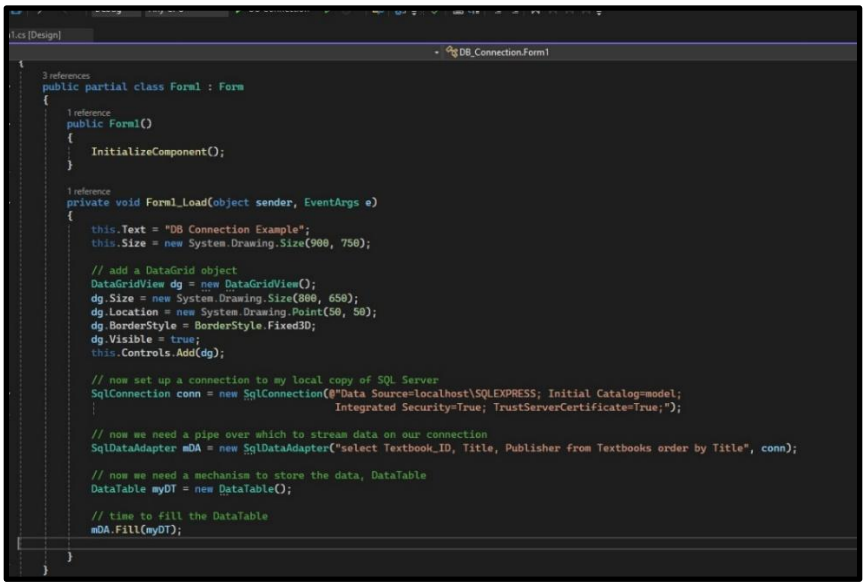
We next need a method for storing our recordset. Our class of interest is the DataTable as shown here. (Figure 20.4) The DataTable object organizes data in rows and columns just like a real table in the database.

Figure 20.4: DataTable



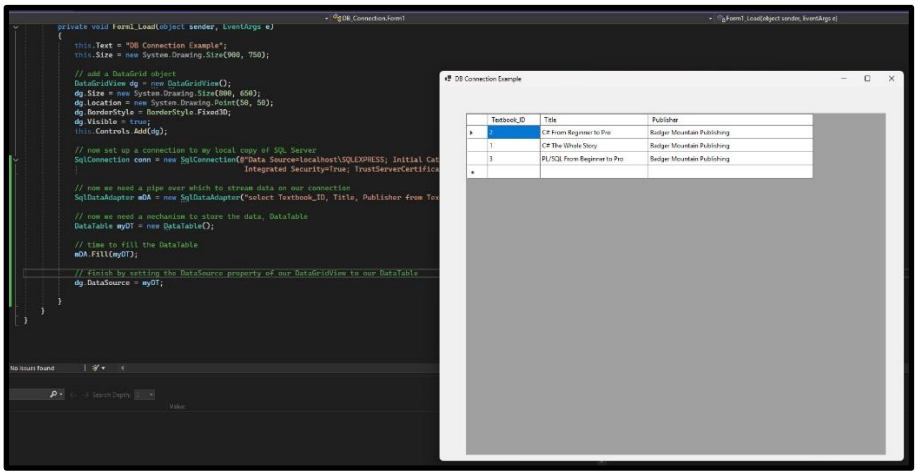
Each SqlDataAdapter class has two methods which you will use all the time; Fill and Push. The Fill method is used to populate a DataTable object with the contents of the SqlDataAdapter data stream. The Push method moves data in the opposite direction, from a DataTable back to the database. (Figure 20.5) We of course will be using the Fill method in our example here.

Figure 20.5: SQLDataAdapter.Fill



We finally get to see some data on the form when we set the DataSource property of our DataGridView to our DataTable. (Figure 20.6) Please note that I didn't specify an order by clause in my SQL command and sure enough the rows were not returned in primary key order. Lesson learned.

Figure 20.6: DataGridView.DataSource



Okay, that puts a wrap on this short lesson. My real goal in this one was to get that connection string working properly. That guy is always tricky. Especially since our audience here is not all using the Express edition on Windows. Lots of other configurations. Let's jump to the outside and review our new classes.

So what did we learn?

- That the DataGridView class is pretty handy for displaying the results of an SQL query. We do so by setting it's DataSource property equal to a DataTable object.
- We then learned about the SQLConnection class. If you got that one to work on the first try then you did better than most because that one varys a bunch based on your SQL Server install.
- Next came the SQLAdapter class which basically defines the SQL command to be executed across your database connection.
- Our next class was the DataTable which stores the results of an SQL query in row by column format.
- Finally we set the DataSource property of our DataGridView to our DataTable object and tahdah, we were looking at information on a C# form which was read from an SQL Server database.

All good stuff. In our next chapter we are going to step up our game by managing information from multiple tables. Should be great fun. Catch you there.